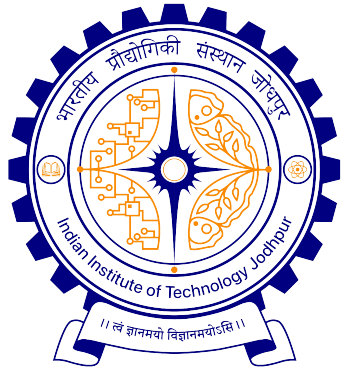




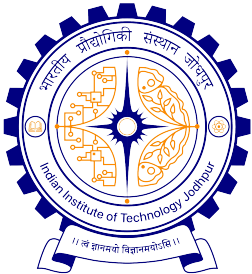
Introduction to Computer Science (CSL1010)

C Basics– Dynamic Memory Allocation L14

Suchetana Chakraborty

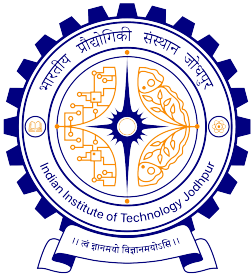
Department of Computer Science, IIT Jodhpur

Spring, 2026



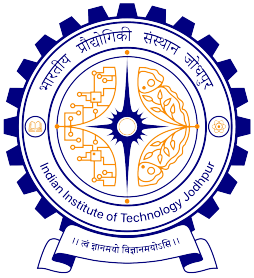
Problems with Array

- Sometimes:
 - Amount of data cannot be predicted beforehand (may be driven by user input).
 - Number of data items keeps changing during program execution.
- Example: Search for an element in an array of N elements
- **One solution:** assume a maximum possible value of N and allocate an array of N elements.
 - Wastes memory space, as N may be much smaller in some executions.
 - Example: maximum value of N may be 10,000, but a particular run may need to search only among 100 elements.
- **Using array of size 10,000 always wastes memory in most cases.**
- On the other extreme, the program cannot handle N larger than 10,000.



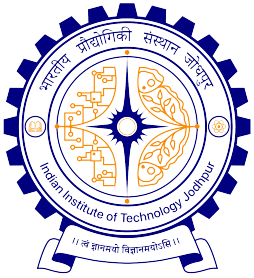
Better Solution

- **Dynamic memory allocation**
 - Know how much memory is needed after the program is run
 - **Example: ask the user to enter from keyboard**
- Dynamically allocate only the amount of memory needed C provides functions to dynamically allocate memory
 - **malloc, calloc, realloc**
- function searches the heap for *size* **contiguous free bytes**
- function returns the address of the first byte
- **programmers responsibility to not lose the pointer**
- **programmers responsibility to not write into area past the last byte allocated**



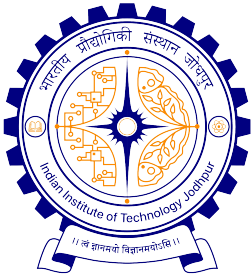
Dynamic Memory Allocation

- Normally the number of elements in an array is pre-specified in the program.
 - Often leads to wastage of memory space or program failure.
- Dynamic Memory Allocation
 - Memory space required can be specified **at the time of execution**.
 - C supports allocating and freeing memory dynamically using library routines.



Memory Allocation Functions

- **malloc**
 - Allocates requested number of bytes and returns a pointer to the first byte of the allocated space.
- **calloc**
 - Allocates space for an array of elements, initializes them to zero and then returns a pointer to the first byte of the memory.
- **free**
 - Frees previously allocated space.
- **realloc**
 - Modifies the size of previously allocated space.



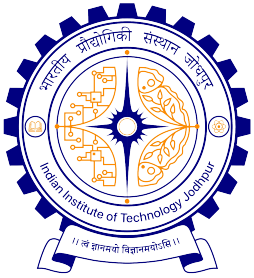
Allocating a Block of Memory

- A block of memory can be allocated using the function `malloc`.
 - Reserves a block of memory of specified size and returns a pointer of type `void *`.
 - The returned pointer can be type-casted to any pointer type.
- General format:

```
ptr = (type *) malloc (byte_size) ;
```

```
int *ptr1 = (int*)malloc(10 * sizeof(int)) ;  
// allocates 10 integers, uninitialized
```

```
int *ptr2 = (int*)calloc(10, sizeof(int)) ;  
// allocates 10 integers, all set to 0
```



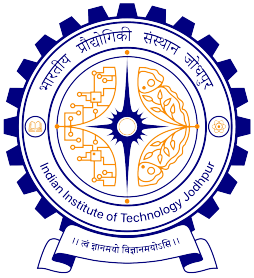
Allocating a Block of Memory

```
p = (int *) malloc(100 * sizeof(int));
```

- A memory space equivalent to (*100 times the size of an int*) bytes is reserved.
- The address of the first byte of the allocated memory is assigned to the pointer **p** of type **int ***.

```
cptr = (char *) malloc (20);
```

- Allocates 20 bytes of space for the pointer **cptr** of type **char ***.

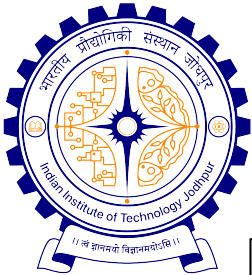


Points to Note

- **malloc** always allocates a block of contiguous bytes.
 - The allocation can fail if sufficient contiguous memory space is not available.
 - If it fails, **malloc** returns **NULL**.

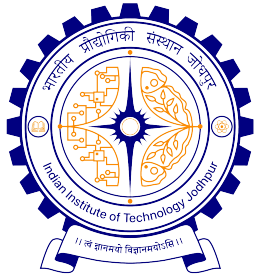
```
if ((p = (int *) malloc(100 * sizeof(int))) == NULL)
{
printf ("Memory cannot be allocated\n");
exit(1);
}
```

- You can use **exit(status)** instead of **return**
- Use **#include <stdlib.h>**.



Example

```
#include <stdio.h>
#include <stdlib.h>
int main () {
    int *ptr, n, i;
    printf("How many integers will you enter? "); scanf("%d", &n);
    if (n <= 0) { printf("Wow! How come?\n"); exit(1); }
    ptr = (int *)malloc(n * sizeof(int));
    if (ptr == NULL) { printf("Oops! I cannot store so many
integers.\n"); exit(2); }
    for (i=0; i<n; ++i) {
        printf("Enter integer no. %d:" , i); scanf("%d", ptr+i);
    }
    ...
    exit(0);
    /* Now, do what you want to do with the integers read and stored in
    ptr[] */
}
```

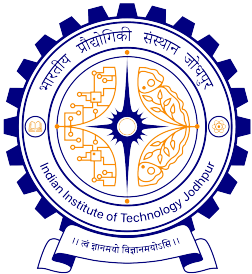


Can we allocate only arrays?

- **malloc** can be used to allocate memory for single variables also:

```
p = (int *) malloc (sizeof(int)) ;
```

- Allocates space for a single int, which can be accessed as ***p** or **p[0]**
- Single variable allocations are just special case of array allocations
- **Array with only one element**
- Single variable allocations are useful for building linked structures as we will see later.



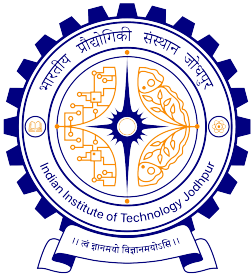
Using malloc for array

- Once the memory is allocated, it can be used with pointers, or with array notation.

- Example:

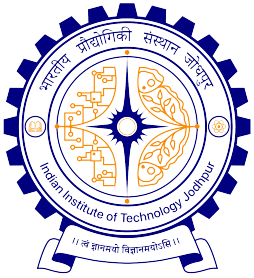
```
int *p, n, i;  
scanf("%d", &n);  
p = (int *) malloc (n * sizeof(int));  
for (i=0; i<n; ++i)  
    scanf("%d", &p[i]);
```

- The n integers allocated can be accessed as `*p`, `*(p+1)`, `*(p+2)`,
`...`, `*(p+n-1)`
- or just as `p[0]`, `p[1]`, `p[2]`, `...`, `p[n-1]`



Releasing the allocated space: free

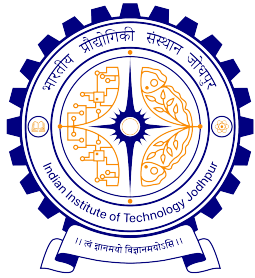
- An allocated block can be returned to the system for future use, by the **free** function.
- General syntax: **free (ptr) ;**
- where **ptr** is a pointer to a memory block which has been previously created using **malloc** (or **calloc** or **realloc**).
- No size is to be mentioned for the allocated block. The system remembers it. The function frees the **entire** block allocated by an earlier **malloc()** type of call.
- **ptr** must be the **starting address** of an allocated block. A pointer to the interior of a block cannot be passed to **free()**.
- Dynamically allocated memory stays until explicitly freed or the program terminates.
- **ptr** must not be null: trying to free the same area twice will generate an error
- You cannot free an array **A[]** defined like this: **int A[50] ;**



Example

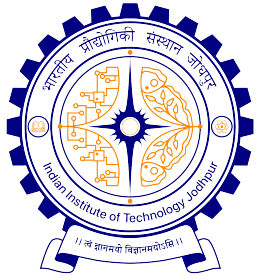
```
int main() {
    int i,N;
    float *height;
    float sum=0,avg;
    printf("Input no. of
students\n");
    scanf("%d", &N);
    height = (float *)
    malloc(N * sizeof(float));
```

```
    printf("Input heights for %d
students\n",N);
    for (i=0; i<N; i++)
        scanf ("%f", &height[i]);
    for(i=0;i<N;i++)
        sum += height[i];
    avg = sum / (float) N;
    printf("Average height =
%f\n", avg);
    free (height);
    return 0;
}
```



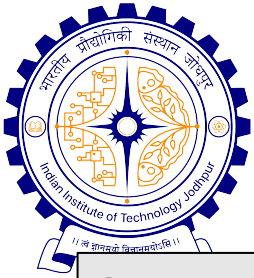
Altering the size of a block

- Sometimes we need to alter the size of some previously allocated memory block.
 - More memory needed.
 - Memory allocated is larger than necessary.
- How?
 - By using the **realloc** function.
- If the original allocation is done as:
ptr = malloc (size) ;
- then reallocation of space may be done as:
ptr = realloc (ptr, newsize) ;



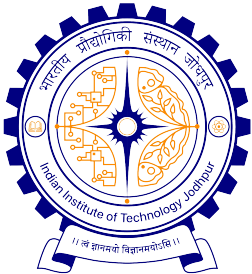
Altering the size of a block... cont.

- The new memory block may or may not begin at the same place as the old one.
 - If it does not find space, it will create an entirely different region and move the contents of the old block into the new block.
- The function guarantees that the old data remains intact.
- If it is unable to allocate, it returns **NULL** and frees the original block.



Example

```
int main () {
    int *A = (int *)malloc(10 * sizeof(int)), allocsize = 10, n = 0, x;
    printf("Keep on entering +ve integers. Enter 0 or a -ve integer to stop.\n");
    while (1) {
        printf("Next integer: "); scanf("%d", &x);
        if (x <= 0) break;
        ++n;
        if (n > allocsize) {
            allocsize += 10;
            A = (int *) realloc(A, allocsize * sizeof(int));
        }
        A[n-1] = x;
    }
    A = (int *) realloc(A, n * sizeof(int)); allocsize = n;
    // Process the integers read from the user
    ...
    free(A);
    return 0;
}
```

Common Mistakes

- Using a pointer before allocating heap space

```
int *ptr;
```

```
*ptr = 5;
```

- Changing the pointer, not the value it references

```
int *ptr = malloc(sizeof(int));
```

```
*ptr = 10; // sets value on stack to 10, not value on the heap
```

- Forgetting to free space on the heap (memory leak)

```
int *p1 = malloc(sizeof(int));
```

```
int *p2 = malloc(sizeof(int));
```

```
p1 = p2; // making p1 point to p2 is fine, but now you can't free the space  
allocated to p1
```

// originally